

10-DAY LIVE COURSE

Frontend Development with React

A day-by-day path from HTML & CSS fundamentals to a deployed, portfolio-ready React application.

Pre-assessment before Day 1 · 2 hours per session

10 sessions

COURSE STRUCTURE

Three phases, ten sessions

An HTML/CSS & JavaScript pre-assessment runs before Day 1 to gauge student baseline and calibrate the pace of the recap sessions.

DAYS 1–2

Foundations

HTML5 & CSS3 recap — semantic structure, forms and media, Flexbox, Grid, and responsive design.

DAYS 3–5

JavaScript

Core language, DOM & events, ES6+ syntax, fetch, and asynchronous JavaScript.

DAYS 6–10

React

Components, props & state, hooks, forms, routing, API integration, and a deployed capstone.

DAY 1 • 2 HOURS

HTML5 & CSS3 Recap — Part 1

01

OF 10 SESSIONS

- 01 Semantic HTML structure and best practices
- 02 Forms, tables, and media embedding
- 03 CSS fundamentals: selectors, box model, typography

Structure pages with meaning, not divs

- Semantic tags describe meaning, not just boxes
- Keep to one `<main>` and one `<h1>` per page
- Screen readers and search engines both benefit

```
<header>
  <nav> ... </nav>
</header>
<main>
  <h1>Page title</h1>
  <p>Welcome to the site.</p>
</main>
<footer>© 2026 My Site</footer>
```

Forms, tables, and media embedding

- Pair every input with a `<label>`
- Tables are for tabular data, not layout
- Images need alt text; video/audio need controls

```
<form>
  <label for="email">Email</label>
  <input id="email" type="email" required>
  <button type="submit">Sign up</button>
</form>

<video src="demo.mp4" controls></video>
```

Selectors, the box model, and type

- Specificity: inline > ID > class > tag
- Box model: content, padding, border, margin
- Set a readable base font-size and line-height

```
.card {  
  padding: 24px;  
  border: 1px solid #e3e6e4;  
  border-radius: 12px;  
}  
.card > h3 {  
  font-size: 20px;  
}
```

DAY 2 • 2 HOURS

HTML5 & CSS3 Recap — Part 2

02

OF 10 SESSIONS

- 01 Flexbox and Grid layout systems
- 02 Responsive design principles and media queries
- 03 Quick recap exercise: rebuild a simple responsive page

Flexbox for rows, Grid for layouts

- Flexbox: one-dimensional, great for toolbars & cards
- Grid: two-dimensional, great for page layout
- Combine both — Grid for structure, Flex inside cells

```
.row {  
  display: flex;  
  gap: 16px;  
}  
  
.page {  
  display: grid;  
  grid-template-columns: 240px 1fr;  
}
```

Media queries & mobile-first design

- Design mobile-first, then add breakpoints
- Use relative units: %, rem, fr — not fixed px widths
- Test at 375px, 768px, and 1280px

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr;  
}  
  
@media (min-width: 768px) {  
  .grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

Recap exercise: rebuild it responsively

- ✓ Start from the provided static mockup
- ✓ Convert the layout to Flexbox and Grid
- ✓ Add a mobile breakpoint at 768px
- ✓ Validate with the browser's device toolbar

DAY 3 • 2 HOURS

JavaScript Fundamentals — Part 1

03

OF 10 SESSIONS

01 Variables, data types, and operators

02 Functions, conditionals, and loops

03 Arrays and objects

Variables, data types & operators

- Use const by default, let when a value changes
- Types: string, number, boolean, null, undefined, object
- typeof checks a value's type at runtime

```
const name = "Ava";  
let age = 27;  
const isStudent = false;  
console.log(typeof age);  
age = age + 1;
```

Functions, conditionals & loops

- Functions package logic you can reuse and name
- if/else branches; switch handles many fixed cases
- for and while repeat work until a condition ends

```
function greet(name) {  
  if (name) {  
    return `Hi, ${name}!`;  
  }  
  return "Hello!";  
}  
  
for (let i = 0; i < 3; i++) {  
  console.log(greet("Ava"));  
}
```

Arrays and objects

- Arrays hold ordered lists; objects hold key/value pairs
- map, filter, and find avoid manual loops
- Dot or bracket notation reads object properties

```
const students = ["Ava", "Ben", "Cy"];
const passed = students.filter(
  (s) => s.length > 2
);
const user = { name: "Ava", age: 27 };
console.log(user.name);
```

DAY 4 • 2 HOURS

JavaScript Fundamentals — Part 2

04

OF 10 SESSIONS

01 DOM selection and manipulation

02 Event handling and form validation

03 ES6+ syntax: arrow functions, destructuring, template literals

Selecting and changing the DOM

- `querySelector` finds one element;
`querySelectorAll` finds many
- `textContent` and `classList` change
what's shown
- Cache the element once, then reuse
the reference

```
const button = document.querySelector("#save");
const label = document.querySelector(".status");
button.addEventListener("click", () => {
  label.textContent = "Saved!";
  label.classList.add("success");
});
```

Event handling & form validation

- `addEventListener` attaches behavior without inline HTML
- `preventDefault` stops a form's default submit
- Validate on submit, then show a clear error message

```
form.addEventListener("submit", (e) => {  
  e.preventDefault();  
  if (!emailInput.value.includes("@")) {  
    showError("Enter a valid email");  
    return;  
  }  
  submitForm();  
});
```

ES6+: arrow functions & destructuring

- Arrow functions keep "this" bound to the outer scope
- Destructuring pulls values out of objects and arrays
- Template literals embed expressions in strings

```
const add = (a, b) => a + b;  
const { name, age } = user;  
const [first, second] = students;  
const msg = `${name} is ${age} years old`;
```

DAY 5 • 2 HOURS

JavaScript Fundamentals — Part 3

05

OF 10 SESSIONS

- 01 Working with fetch and APIs
- 02 Asynchronous JavaScript: promises and async/await
- 03 Mini exercise: build a small interactive JS widget

Fetching data from an API

- `fetch()` returns a Promise that resolves to a Response
- Call `.json()` to parse the response body
- Always handle the error path, not just the happy path

```
fetch("https://api.example.com/users")  
  .then((res) => res.json())  
  .then((data) => renderUsers(data))  
  .catch((err) => showError(err));
```

Promises & async/await

- async/await reads like synchronous code
- try/catch handles rejected promises
- await only pauses inside an async function

```
async function loadUsers() {  
  try {  
    const res = await fetch("/api/users");  
    const data = await res.json();  
    renderUsers(data);  
  } catch (err) {  
    showError(err);  
  }  
}
```

Mini exercise: an interactive widget

✓ Fetch data from a public API on page load

✓ Render the results into the DOM

✓ Add a loading state and an error state

✓ Let the user filter or refresh the results

DAY 6 • 2 HOURS

Introduction to React

06

OF 10 SESSIONS

01 Why React; component-based architecture

02 Setting up a React project (Vite/CRA)

03 JSX syntax and rendering components

Components as building blocks

- UI is broken into small, reusable components
- Each component owns its own markup, logic & state
- React re-renders efficiently when data changes

```
<App>
  <Header />
  <MainContent>
    <Card />
    <Card />
  </MainContent>
  <Footer />
</App>
```

Setting up a React project

- Vite gives a fast dev server and build out of the box
- `npm run dev` starts local development with hot reload
- Components live under `src/components`

```
$ npm create vite@latest my-app -- --template react
$ cd my-app
$ npm install
$ npm run dev
```

JSX: writing markup in JavaScript

- JSX compiles to `React.createElement` calls
- Curly braces embed any JavaScript expression
- Every component returns exactly one root element

```
function Welcome({ name }) {  
  return (  
    <div className="welcome">  
      <h1>Hello, {name}!</h1>  
    </div>  
  );  
}
```

DAY 7 • 2 HOURS

React Props & State

07

OF 10 SESSIONS

- 01 Passing data with props
- 02 Managing component state with useState
- 03 Handling events in React

Passing data down with props

- Props flow one direction: parent to child
- A component is just a function of its props
- Destructure props for cleaner component code

```
function Card({ title, price }) {  
  return (  
    <div className="card">  
      <h3>{title}</h3>  
      <p>${price}</p>  
    </div>  
  );  
}  
  
<Card title="Mug" price={12} />
```

Managing state with useState

- useState returns a value and a setter function
- Calling the setter triggers a re-render
- Never mutate state directly — always use the setter

```
import { useState } from "react";  
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      Clicked {count} times  
    </button>  
  );  
}
```

Handling events in React

- Event props use camelCase: `onClick`, `onChange`
- Pass a function reference, not a function call
- The handler receives a React synthetic event

```
function SearchBox() {  
  const handleChange = (e) => {  
    console.log(e.target.value);  
  };  
  return <input onChange={handleChange} />;  
}
```

DAY 8 • 2 HOURS

React Hooks & Component Composition

08

OF 10 SESSIONS

01 useEffect and side effects

02 Component composition and reusable components

03 Conditional rendering and lists (map, keys)

Side effects with useEffect

- useEffect runs after the component renders
- The dependency array controls when it re-runs
- Return a cleanup function to avoid leaks

```
useEffect(() => {  
  const id = setInterval(() => tick(), 1000);  
  return () => clearInterval(id);  
}, []);
```

Composition over one giant component

- Split UI into small, focused, reusable pieces
- children lets a component wrap arbitrary content
- Compose small components instead of duplicating markup

```
function Panel({ title, children }) {  
  return (  
    <section className="panel">  
      <h3>{title}</h3>  
      {children}  
    </section>  
  );  
}  
  
<Panel title="Profile"><Avatar /></Panel>
```

Conditional rendering & lists

- `&&` renders only when the condition is true
- `.map()` turns an array into a list of elements
- Every list item needs a stable, unique key

```
{isLoading && <Spinner />}  
<ul>  
  {users.map((u) => (  
    <li key={u.id}>{u.name}</li>  
  ))}  
</ul>
```

DAY 9 • 2 HOURS

React Forms, Routing & API Integration

09

OF 10 SESSIONS

- 01 Controlled forms in React
- 02 Client-side routing with React Router
- 03 Fetching and displaying API data in a React app

Controlled forms in React

- The input's value comes from state, not the DOM
- `onChange` updates state on every keystroke
- One state object can hold an entire form

```
const [email, setEmail] = useState("");  
<input  
  value={email}  
  onChange={(e) => setEmail(e.target.value)}  
>
```

Client-side routing with React Router

- Routes map a URL path to a component
- Link navigates without a full page reload
- Nested routes build layouts with shared chrome

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
</Routes>
<Link to="/about">About</Link>
```

Fetching & displaying API data

- Fetch inside `useEffect`, store results in state
- Show a loading state while the request is in flight
- Render the list once data arrives

```
useEffect(() => {  
  fetch("/api/products")  
    .then((res) => res.json())  
    .then(setProducts);  
}, []);
```

DAY 10 • 2 HOURS

Capstone Project & Deployment

10

OF 10 SESSIONS

- 01 Building out the final portfolio project
- 02 Styling and responsiveness review
- 03 Deploying the React project (Vercel/Netlify) and presentation

Capstone project requirements

- ✓ At least 3 connected React components
- ✓ One page that fetches and displays real API data
- ✓ Client-side routing between 2+ views
- ✓ A polished, personal portfolio topic

Styling & responsiveness review

✓ Consistent spacing, type scale, and color palette

✓ Layout works at 375px, 768px, and 1280px

✓ Interactive states: hover, focus, and disabled

✓ No console errors or warnings before shipping

Deploying & presenting the project

- Push the project to GitHub before deploying
- Vercel and Netlify both deploy straight from a repo
- Practice a 3-minute walkthrough of the live app

```
$ npm run build  
$ git push origin main  
$ vercel --prod
```

DAY 10 • WRAP-UP

Ship it, present it, own it.

Every student leaves with a deployed, responsive React portfolio project — and the fundamentals to keep building.

10

Sessions, 2 hours each

3

Phases: HTML/CSS, JS, React

1

Deployed capstone project

Thank you

Frontend Development with React